

Математическое моделирование

Лабораторная работа № 7

Метвалли Ахмед Фарг Набеев

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Теоретические сведения	8
3.2	Задача	11
4	Численное решение трёх задач Коши и визуализация	13
4.1	Инициализация проекта и загрузка пакетов	13
4.2	Общие начальные условия	14
4.3	Первая модель	14
4.4	Вторая модель	14
4.5	Третья модель	15
4.6	Утилита: один прогон модели	16
4.7	Запуск первой модели	17
4.8	Запуск второй модели	18
4.9	Запуск третьей модели	18
4.10	Вывод первых строк таблиц	19
4.11	Показать график в интерактивной среде	19
5	Параметрическое исследование трёх дифференциальных моделей	20
5.1	Активация проекта и загрузка пакетов	20
5.2	Установка каталогов	20
5.3	Определение моделей	21
5.4	Выбор модели по типу эксперимента	21
5.5	Правая часть уравнения для анализа производной	22
5.6	Базовые параметры первой модели	22
5.7	Базовые параметры второй модели	23
5.8	Базовые параметры третьей модели	24
5.9	Функция для запуска одного эксперимента	25
5.10	Функция печати результатов одного эксперимента	28
5.11	Функция построения графика $n(t)$	29
5.12	Функция построения графика производной dn/dt	30
5.13	Функция построения фазовой траектории dn/dt от n	31
5.14	Запуск базовых экспериментов	32

5.15	Параметрическое сканирование	33
5.16	Запуск всех экспериментов	36
5.17	Анализ результатов сканирования	39
5.18	Сравнительные графики $n(t)$ для каждой модели	39
5.19	Сравнительные графики dn/dt для каждой модели	41
5.20	Сравнительные фазовые траектории $dn/dt(n)$	42
5.21	График зависимости n_final от параметра a	44
5.22	График зависимости n_final от параметра b	45
5.23	График зависимости n_max от параметра a	46
5.24	График зависимости n_max от параметра b	47
5.25	График зависимости $saturation_final$ от параметра a	48
5.26	График зависимости $saturation_final$ от параметра b	49
5.27	Бенчмаркинг	50
5.28	График времени вычисления от параметра a	52
5.29	График времени вычисления от параметра b	53
5.30	Сохранение всех результатов	54
5.31	Базовые эксперименты	55
5.32	Сравнение базовых экспериментов	66
5.33	Параметрическое сканирование	67
5.34	Бенчмаркинг	74
5.35	Выводы	77

Список литературы

80

Список иллюстраций

3.1	График решения уравнения модели Мальтуса	10
3.2	График логистической кривой	11
5.1	Первая модель: зависимость $n(t)$	55
5.2	Первая модель: скорость изменения	56
5.3	Первая модель: фазовая траектория	57
5.4	Вторая модель: зависимость $n(t)$	58
5.5	Вторая модель: скорость изменения	59
5.6	Вторая модель: фазовая траектория	60
5.7	Третья модель: зависимость $n(t)$	63
5.8	Третья модель: скорость изменения	64
5.9	Третья модель: фазовая траектория	65
5.10	Зависимость итогового значения n от параметра a	67
5.11	Зависимость итогового значения n от параметра b	68
5.12	Зависимость максимального значения n от параметра a	69
5.13	Зависимость максимального значения n от параметра b	71
5.14	Зависимость финального насыщения от параметра a	72
5.15	Зависимость финального насыщения от параметра b	73
5.16	Зависимость времени решения ODE от параметра a	74
5.17	Зависимость времени решения ODE от параметра b	76

Список таблиц

1 Цель работы

Исследовать математическую модель эффективности рекламной кампании.

2 Задание

1. Изучить модель распространения рекламы.
2. Построить графики изменения числа информированных покупателей для заданных случаев.
3. Для второго случая определить момент времени, когда скорость распространения рекламы достигает максимума.

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассматривается рекламная кампания, направленная на продвижение нового товара или услуги. Главная практическая цель состоит в том, чтобы будущая прибыль от продаж превысила расходы на рекламу. В начале кампании затраты могут быть выше прибыли, поскольку о продукте знает лишь небольшая часть потенциальной аудитории. По мере распространения информации число покупателей растёт, прибыль увеличивается, а затем рынок постепенно приближается к насыщению. После этого дальнейшее рекламирование теряет эффективность.

Пусть торговая организация реализует некоторый товар. В момент времени t из общего числа потенциальных покупателей N о товаре знают $n(t)$ человек. Для увеличения продаж запускается реклама через радио, телевидение и другие каналы массовой коммуникации. После начала кампании сведения о товаре распространяются не только через рекламные объявления, но и через общение покупателей между собой.

Обозначим:

- $\frac{dn}{dt}$ — скорость изменения числа потребителей, узнавших о товаре и готовых приобрести его;
- t — время, прошедшее с начала рекламной кампании;
- N — общее количество потенциальных платёжеспособных покупателей;

- $n(t)$ — число уже информированных клиентов.

Вклад внешней рекламы пропорционален количеству покупателей, которые ещё не знают о товаре. Он описывается выражением

$$\alpha_1(t)(N - n(t)),$$

где $\alpha_1(t) > 0$ характеризует интенсивность рекламной кампании и зависит от текущих затрат на продвижение.

Кроме внешней рекламы, информацию распространяют уже осведомлённые покупатели. Этот механизм соответствует «сарафанному радио» и задаётся выражением

$$\alpha_2(t)n(t)(N - n(t)).$$

Чем больше покупателей уже знают о товаре, тем сильнее вклад данного механизма в дальнейшее распространение информации.

В результате математическая модель рекламной кампании записывается в виде

$$\frac{dn}{dt} = (\alpha_1(t) + \alpha_2(t)n(t))(N - n(t)).$$

Если $\alpha_1(t) \gg \alpha_2(t)$, то основную роль играет внешняя реклама. В этом режиме модель близка к модели Мальтуса.

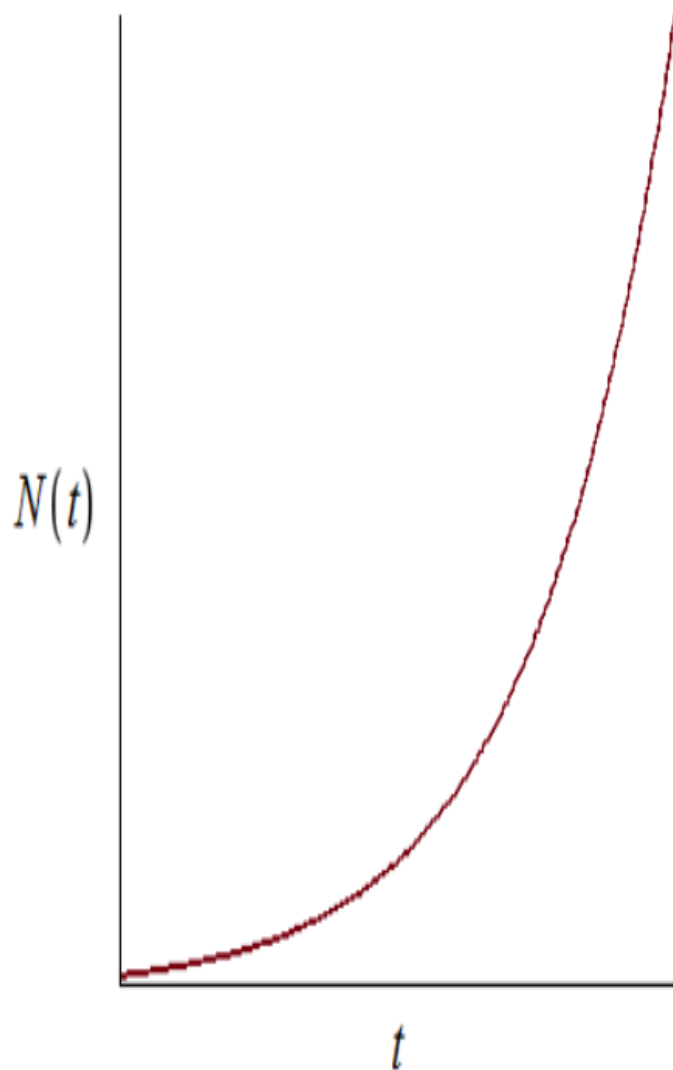


Рисунок 3.1: График решения уравнения модели Мальтуса

Если $\alpha_1(t) \ll \alpha_2(t)$, то решающее влияние оказывает распространение информации между покупателями. В этом случае динамика приближается к логистической модели роста.

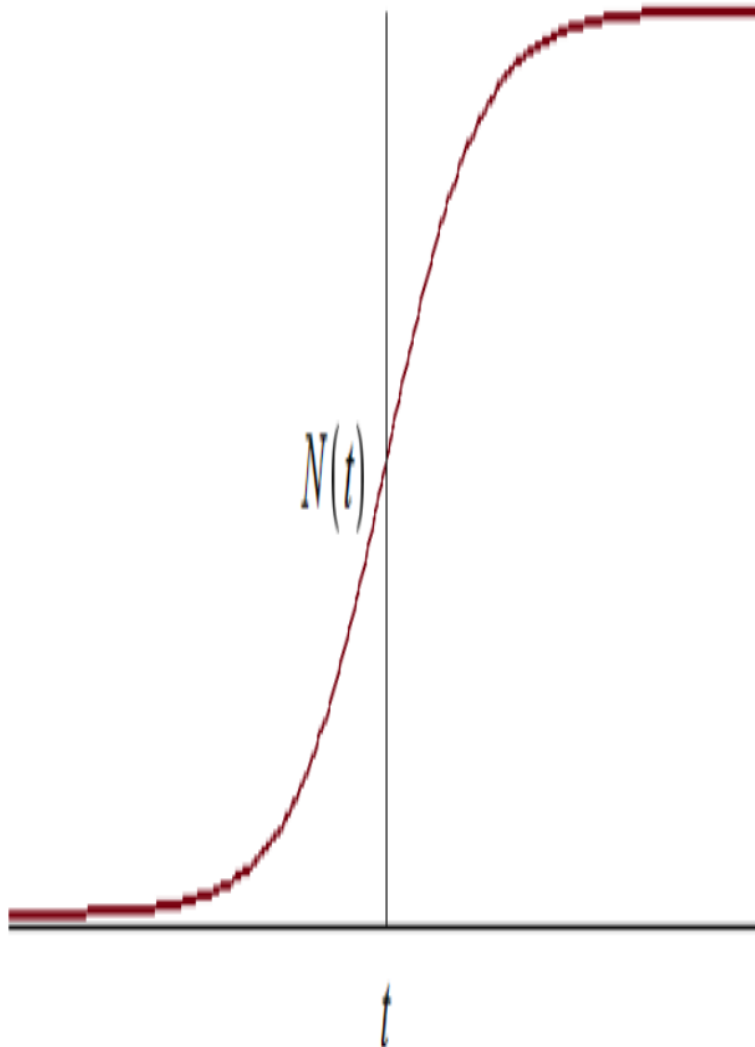


Рисунок 3.2: График логистической кривой

3.2 Задача

Необходимо построить графики распространения рекламы для моделей:

1.

$$\frac{dn}{dt} = (0.54 + 0.00016n(t))(N - n(t)).$$

2.

$$\frac{dn}{dt} = (0.000021 + 0.38n(t))(N - n(t)).$$

3.

$$\frac{dn}{dt} = (0.2 \cos t + 0.2 \cos(2t)n(t))(N - n(t)).$$

Объём потенциальной аудитории равен $N = 609$. В начальный момент времени о товаре знают 4 человека, то есть

$$n(0) = 4.$$

Для второго случая требуется найти момент времени, при котором скорость распространения рекламы принимает максимальное значение.

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

4 Численное решение трёх задач

Коши и визуализация

Цель: решить три варианта дифференциального уравнения, построить графики $n(t)$, сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Общие начальные условия

```
N = 609.0  
n0 = 4.0  
u0 = [n0]
```

4.3 Первая модель

Уравнение: $dn/dt = (a + bn)(N - n)$

```
a1 = 0.54  
b1 = 0.00016  
  
t0_1 = 0.0  
tmax_1 = 10.0  
tspan_1 = (t0_1, tmax_1)  
  
p1 = (a = a1, b = b1, N = N)  
  
function model_case_1!(dy, y, p, t)  
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])  
end
```

4.4 Вторая модель

Уравнение: $dn/dt = (a + bn)(N - n)$

```

a2 = 0.000021
b2 = 0.38

t0_2 = 0.0
tmax_2 = 0.04
tspan_2 = (t0_2, tmax_2)

p2 = (a = a2, b = b2, N = N)

function model_case_2!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end

```

4.5 Третья модель

Уравнение: $dn/dt = (a \cos(t) + b \cos(2t)n) * (N - n)$

```

a3 = 0.2
b3 = 0.2

t0_3 = 0.0
tmax_3 = 0.15
tspan_3 = (t0_3, tmax_3)

p3 = (a = a3, b = b3, N = N)

function model_case_3!(dy, y, p, t)
    dy[1] = (p.a * cos(t) + p.b * cos(2 * t) * y[1]) * (p.N - y[1])
end

```

```
end
```

4.6 Утилита: один прогон модели

```
function run_case(case_name; model!, u0, tspan, p, nt = 500, image_name = case_name)
```

— Сетка по времени —

```
tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
prob = ODEProblem(model!, u0, tspan, p)  
sol = solve(prob, Tsit5(), saveat = tgrid)
```

— Таблица с результатами —

```
df = DataFrame(  
    t = sol.t,  
    n = [u[1] for u in sol.u]  
)
```

— График $n(t)$ —

```
plt = plot(  
    sol,  
    xlabel = "t",  
    ylabel = "n(t)",  
    label = "n(t)",  
    lw = 2,
```



```

        title = "Численное решение – $case_name",
        legend = :topright
    )

```

— Сохранение графика —

```

savefig(plt, plotsdir(script_name, "$(image_name).png"))

```

— Сохранение таблицы —

```

CSV.write(datadir(script_name, "table_$(case_name).csv"), df)

```

— Сохранение данных в JLD2 —

```

@save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan nt

    return (
        sol = sol,
        df = df,
        plt = plt
    )
end

```

4.7 Запуск первой модели

```

res_1 = run_case(
    "case_1";
    model! = model_case_1!,
    u0 = u0,

```

```
tspan = tspan_1,  
p = p1,  
nt = 500,  
image_name = "04"  
)
```

4.8 Запуск второй модели

```
res_2 = run_case(  
    "case_2";  
    model! = model_case_2!,  
    u0 = u0,  
    tspan = tspan_2,  
    p = p2,  
    nt = 500,  
    image_name = "05"  
)
```

4.9 Запуск третьей модели

```
res_3 = run_case(  
    "case_3";  
    model! = model_case_3!,  
    u0 = u0,  
    tspan = tspan_3,  
    p = p3,
```

```
nt = 500,  
image_name = "06"  
)
```

4.10 Вывод первых строк таблиц

```
println("Первые 5 строк таблицы результатов для первой модели:")  
println(first(res_1.df, 5))  
  
println()  
println("Первые 5 строк таблицы результатов для второй модели:")  
println(first(res_2.df, 5))  
  
println()  
println("Первые 5 строк таблицы результатов для третьей модели:")  
println(first(res_3.df, 5))
```

4.11 Показать график в интерактивной среде

```
res_1.plt
```

5 Параметрическое исследование трёх дифференциальных моделей

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
using Statistics
```

5.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.3 Определение моделей

Первая модель: $dn/dt = (a + bn)(N - n)$

```
function model_case_1!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end
```

Вторая модель: $dn/dt = (a + bn)(N - n)$

```
function model_case_2!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end
```

Третья модель: $dn/dt = (a \cos(t) + b \cos(2t)n)(N - n)$

```
function model_case_3!(dy, y, p, t)
    dy[1] = (p.a * cos(t) + p.b * cos(2 * t) * y[1]) * (p.N - y[1])
end
```

5.4 Выбор модели по типу эксперимента

```
function choose_model(model_type)
    if model_type == :case1
        return model_case_1!
    elseif model_type == :case2
        return model_case_2!
    elseif model_type == :case3
        return model_case_3!
    else
    end
end
```

```

        error("Неизвестный тип модели: $model_type")
    end
end

```

5.5 Правая часть уравнения для анализа производной

```

function rhs_value(model_type, n, p, t)
    if model_type == :case1
        return (p.a + p.b * n) * (p.N - n)
    elseif model_type == :case2
        return (p.a + p.b * n) * (p.N - n)
    elseif model_type == :case3
        return (p.a * cos(t) + p.b * cos(2 * t) * n) * (p.N - n)
    else
        error("Неизвестный тип модели: $model_type")
    end
end

```

5.6 Базовые параметры первой модели

```

base_params1 = Dict(
    :N => 609.0,
    :n0 => 4.0,

    :tspan => (0.0, 10.0),
    :nt => 500,

```

```

:model_type => :case1,

:a => 0.54,
:b => 0.00016,

:solver => Tsit5(),
:experiment_name => "base_experiment_case1"
)

```

5.7 Базовые параметры второй модели

```

base_params2 = Dict(
  :N => 609.0,
  :n0 => 4.0,

  :tspan => (0.0, 0.04),
  :nt => 500,

  :model_type => :case2,

  :a => 0.000021,
  :b => 0.38,

  :solver => Tsit5(),
  :experiment_name => "base_experiment_case2"
)

```

5.8 Базовые параметры третьей модели

```
base_params3 = Dict(
    :N => 609.0,
    :n0 => 4.0,

    :tspan => (0.0, 0.15),
    :nt => 500,

    :model_type => :case3,

    :a => 0.2,
    :b => 0.2,

    :solver => Tsit5(),
    :experiment_name => "base_experiment_case3"
)

base_experiments = [base_params1, base_params2, base_params3]

println("Базовые параметры экспериментов:")
for params in base_experiments
    println("\nmodel_type = ", params[:model_type])
    println(" N = ", params[:N])
    println(" n0 = ", params[:n0])
    println(" tspan = ", params[:tspan])
    println(" nt = ", params[:nt])
    println(" a = ", params[:a])
end
```



```
println(" b = ", params[:b])  
end
```

5.9 Функция для запуска одного эксперимента

```
function run_single_experiment(params::Dict)  
    N = params[:N]  
    n0 = params[:n0]  
  
    tspan = params[:tspan]  
    nt = params[:nt]  
  
    model_type = params[:model_type]  
    model! = choose_model(model_type)  
  
    a = params[:a]  
    b = params[:b]  
  
    solver = params[:solver]  
  
    u0 = [n0]  
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))  
  
    p = (a = a, b = b, N = N)  
  
    prob = ODEProblem(model!, u0, tspan, p)  
    sol = solve(prob, solver; saveat = tgrid)
```

```

t_vals = sol.t
n_vals = [u[1] for u in sol.u]
dn_vals = [rhs_value(model_type, n_vals[i], p, t_vals[i]) for i in eachindex(t_vals)]

```

— Мини-анализ —

```

n_initial = n_vals[1]
n_final = n_vals[end]

n_max = maximum(n_vals)
n_min = minimum(n_vals)
n_mean = mean(n_vals)

dn_final = dn_vals[end]
dn_max = maximum(dn_vals)
dn_min = minimum(dn_vals)
dn_mean = mean(dn_vals)

growth_abs = n_final - n_initial
growth_rel = growth_abs / n_initial

saturation_final = n_final / N
saturation_max = n_max / N

idx_n_max = argmax(n_vals)
t_at_n_max = t_vals[idx_n_max]

idx_90 = findfirst(x -> x >= 0.9 * N, n_vals)
t_reach_90 = isnothing(idx_90) ? missing : t_vals[idx_90]

```

```

idx_95 = findfirst(x -> x >= 0.95 * N, n_vals)
t_reach_95 = isnothing(idx_95) ? missing : t_vals[idx_95]

return Dict(
  "solution" => sol,
  "t_points" => t_vals,

  "n_values" => n_vals,
  "dn_values" => dn_vals,

  "n_initial" => n_initial,
  "n_final" => n_final,

  "n_max" => n_max,
  "n_min" => n_min,
  "n_mean" => n_mean,

  "dn_final" => dn_final,
  "dn_max" => dn_max,
  "dn_min" => dn_min,
  "dn_mean" => dn_mean,

  "growth_abs" => growth_abs,
  "growth_rel" => growth_rel,

  "saturation_final" => saturation_final,
  "saturation_max" => saturation_max,

```

```

        "t_at_n_max" => t_at_n_max,
        "t_reach_90" => t_reach_90,
        "t_reach_95" => t_reach_95,

        "parameters" => params
    )
end

```

5.10 Функция печати результатов одного эксперимента

```

function print_experiment_summary(data, path)
    println(" n_initial: ", data["n_initial"])
    println(" n_final: ", data["n_final"])
    println(" n_min: ", data["n_min"])
    println(" n_max: ", data["n_max"])
    println(" n_mean: ", round(data["n_mean"]; digits = 4))

    println(" dn_final: ", data["dn_final"])
    println(" dn_min: ", data["dn_min"])
    println(" dn_max: ", data["dn_max"])
    println(" dn_mean: ", round(data["dn_mean"]; digits = 4))

    println(" growth_abs: ", data["growth_abs"])
    println(" growth_rel: ", round(data["growth_rel"]; digits = 4))

    println(" saturation_final: ", round(data["saturation_final"]; digits = 4))

```

```

println(" saturation_max: ", round(data["saturation_max"]; digits = 4))

println(" t_at_n_max: ", data["t_at_n_max"])
println(" t_reach_90: ", data["t_reach_90"])
println(" t_reach_95: ", data["t_reach_95"])

println(" Файл результатов: ", path)
end

```

5.11 Функция построения графика $n(t)$

```

function plot_time_series(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["t_points"],
        data["n_values"],
        xlabel = "t",
        ylabel = "n(t)",
        label = "n(t)",
        title = "Динамика n(t): model_type=$model_type",
        lw = 2,
        legend = :bottomright,
        grid = true
    )

    hline!(

```

```

        plt,
        [params[:N]],
        label = "N",
        lw = 2,
        linestyle = :dash
    )

    return plt
end

```

5.12 Функция построения графика производной dn/dt

```

function plot_derivative_series(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["t_points"],
        data["dn_values"],
        xlabel = "t",
        ylabel = "dn/dt",
        label = "dn/dt",
        title = "Скорость изменения: model_type=$model_type",
        lw = 2,
        legend = :topright,
        grid = true
    )
end

```

```
    return plt
end
```

5.13 Функция построения фазовой траектории dn/dt от n

```
function plot_phase_n_dn(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["n_values"],
        data["dn_values"],
        xlabel = "n",
        ylabel = "dn/dt",
        label = "Фазовая траектория",
        title = "Фазовая траектория dn/dt(n): model_type=$model_type",
        lw = 2,
        legend = :topright,
        grid = true
    )

    return plt
end
```

5.14 Запуск базовых экспериментов

```
base_results = Dict()

for params in base_experiments
    model_type = params[:model_type]

    println("\n" * "="^60)
    println("БАЗОВЫЙ ЭКСПЕРИМЕНТ: model_type=$model_type")
    println("="^60)

    data, path = produce_or_load(
        datadir(script_name, "single"),
        params,
        run_single_experiment;
        prefix = "model",
        tag = false,
        verbose = true
    )

    base_results[model_type] = Dict(
        "data" => data,
        "path" => path
    )

    print_experiment_summary(data, path)

    plt_time = plot_time_series(data, params)
```



```

savefig(plt_time, plotsdir(script_name, "single_experiment_$(model_type).png"))

plt_derivative = plot_derivative_series(data, params)
savefig(plt_derivative, plotsdir(script_name, "single_experiment_$(model_type)_de

plt_phase = plot_phase_n_dn(data, params)
savefig(plt_phase, plotsdir(script_name, "single_experiment_$(model_type)_phase_n

df_base = DataFrame(
    t = data["t_points"],
    n = data["n_values"],
    dn = data["dn_values"],
    model_type = fill(string(model_type), length(data["t_points"])),
    a = fill(params[:a], length(data["t_points"])),
    b = fill(params[:b], length(data["t_points"])),
    N = fill(params[:N], length(data["t_points"]))
)

CSV.write(datadir(script_name, "table_single_$(model_type).csv"), df_base)
end

```

5.15 Параметрическое сканирование

Сетка параметров для первой модели

```

param_grid_case1 = Dict(
    :N => [609.0],
    :n0 => [4.0],

```

```

:tspan => [(0.0, 10.0)],
:nt => [500],

:model_type => [:case1],

:a => [0.35, 0.54, 0.70],
:b => [0.00008, 0.00016, 0.00024],

:solver => [Tsit5()],
:experiment_name => ["parametric_scan_case1"]
)

```

Сетка параметров для второй модели

```

param_grid_case2 = Dict(
  :N => [609.0],
  :n0 => [4.0],

  :tspan => [(0.0, 0.04)],
  :nt => [500],

  :model_type => [:case2],

  :a => [0.00001, 0.000021, 0.00004],
  :b => [0.25, 0.38, 0.50],

  :solver => [Tsit5()],
  :experiment_name => ["parametric_scan_case2"]
)

```

Сетка параметров для третьей модели

```
param_grid_case3 = Dict(
    :N => [609.0],
    :n0 => [4.0],

    :tspan => [(0.0, 0.15)],
    :nt => [500],

    :model_type => [:case3],

    :a => [0.10, 0.20, 0.30],
    :b => [0.10, 0.20, 0.30],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan_case3"]
)

all_params = vcat(
    dict_list(param_grid_case1),
    dict_list(param_grid_case2),
    dict_list(param_grid_case3)
)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("="^60)
```

5.16 Запуск всех экспериментов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println(
        "Порядок: $i/${length(all_params)} | ",
        "model_type=$(params[:model_type]) | ",
        "a=$(params[:a]) | b=$(params[:b])"
    )

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    result_summary = (
        model_type = string(params[:model_type]),

        N = params[:N],
        n0 = params[:n0],

        t_start = params[:tspan][1],
```

```

t_end = params[:tspan][2],
nt = params[:nt],

a = params[:a],
b = params[:b],

n_initial = data["n_initial"],
n_final = data["n_final"],

n_min = data["n_min"],
n_max = data["n_max"],
n_mean = data["n_mean"],

dn_final = data["dn_final"],
dn_min = data["dn_min"],
dn_max = data["dn_max"],
dn_mean = data["dn_mean"],

growth_abs = data["growth_abs"],
growth_rel = data["growth_rel"],

saturation_final = data["saturation_final"],
saturation_max = data["saturation_max"],

t_at_n_max = data["t_at_n_max"],
t_reach_90 = data["t_reach_90"],
t_reach_95 = data["t_reach_95"],

```

```

        filepath = path
    )

    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        n = data["n_values"],
        dn = data["dn_values"],

        model_type = fill(string(params[:model_type]), length(data["t_points"])),

        N = fill(params[:N], length(data["t_points"])),
        n0 = fill(params[:n0], length(data["t_points"])),

        a = fill(params[:a], length(data["t_points"])),
        b = fill(params[:b], length(data["t_points"])),

        t_start = fill(params[:tspan][1], length(data["t_points"])),
        t_end = fill(params[:tspan][2], length(data["t_points"]))
    )

    push!(all_dfs, df)
end

```

5.17 Анализ результатов сканирования

```
results_df = DataFrame(all_results)
full_timeseries_df = vcat(all_dfs...)

println("\nСводная таблица результатов:")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)
CSV.write(datadir(script_name, "timeseries_full.csv"), full_timeseries_df)
```

5.18 Сравнительные графики $n(t)$ для каждой модели

```
model_types = unique(results_df.model_type)

for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) == model_type_string

    plt = plot(size = (950, 520), dpi = 150)

    for params in params_subset
        data, _ = produce_or_load(
            datadir(script_name, "parametric_scan"),
            params,
            run_single_experiment;
            prefix = "scan",
            tag = false,
```

```

        verbose = false
    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["t_points"],
        data["n_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    plt,
    xlabel = "t",
    ylabel = "n(t)",
    title = "Сканирование: траектории n(t), model_type=model_type_string",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_n_comparison_$(model_type_str
end

```


5.19 Сравнительные графики dn/dt для каждой модели

```
for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) == model_type_string

plt = plot(size = (950, 520), dpi = 150)

for params in params_subset
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["t_points"],
        data["dn_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
```

```

end

plot!(
    plt,
    xlabel = "t",
    ylabel = "dn/dt",
    title = "Сканирование: скорость изменения, model_type=$model_type_string",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_dn_comparison_$(model_type_st
end

```

5.20 Сравнительные фазовые траектории $dn/dt(n)$

```

for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) == model_type_string

    plt = plot(size = (950, 520), dpi = 150)

    for params in params_subset
        data, _ = produce_or_load(
            datadir(script_name, "parametric_scan"),
            params,
            run_single_experiment;
            prefix = "scan",

```

```

        tag = false,
        verbose = false
    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["n_values"],
        data["dn_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    plt,
    xlabel = "n",
    ylabel = "dn/dt",
    title = "Сканирование: фазовые траектории dn/dt(n), model_type=$model_type_str",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_phase_n_dn_$(model_type_str)"),
end

```

5.21 График зависимости n_{final} от параметра a

```
p_a_final = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type == model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_final,
            sub.a,
            sub.n_final,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\text{final}}$  от  $a$ "
        )
    end
end

plot!(
    p_a_final,
    xlabel = "a",
    ylabel = " $n_{\text{final}}$ ",
    title = "Зависимость итогового значения  $n$  от параметра  $a$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_a_final, plotsdir(script_name, "n_final_vs_a.png"))
```

5.22 График зависимости n_{final} от параметра b

```
p_b_final = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type == model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_final,
            sub.b,
            sub.n_final,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\text{final}}$  от  $b$ "
        )
    end
end

plot!(
    p_b_final,
    xlabel = "b",
    ylabel = " $n_{\text{final}}$ ",
    title = "Зависимость итогового значения  $n$  от параметра  $b$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_b_final, plotsdir(script_name, "n_final_vs_b.png"))
```

5.23 График зависимости $n_{\max}$ от параметра a

```
p_a_max = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_max,
            sub.a,
            sub.n_max,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\max}$  от  $a$ "
        )
    end
end

plot!(
    p_a_max,
    xlabel = "a",
    ylabel = " $n_{\max}$ ",
    title = "Зависимость максимального значения  $n$  от параметра  $a$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_a_max, plotsdir(script_name, "n_max_vs_a.png"))
```

5.24 График зависимости $n_{\max}$ от параметра b

```
p_b_max = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_max,
            sub.b,
            sub.n_max,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\max}$  от  $b$ "
        )
    end
end

plot!(
    p_b_max,
    xlabel = "b",
    ylabel = " $n_{\max}$ ",
    title = "Зависимость максимального значения  $n$  от параметра  $b$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_b_max, plotsdir(script_name, "n_max_vs_b.png"))
```

5.25 График зависимости `saturation_final` от параметра `a`

```
p_a_saturation = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type == model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_saturation,
            sub.a,
            sub.saturation_final,
            seriestype = :scatter,
            label = "$model_type_string: saturation_final от a"
        )
    end
end

plot!(
    p_a_saturation,
    xlabel = "a",
    ylabel = "n_final / N",
    title = "Зависимость финального насыщения от параметра a",
    legend = :bottomright,
    grid = true
)
```



```
savefig(p_a_saturation, plotsdir(script_name, "saturation_final_vs_a.png"))
```

5.26 График зависимости `saturation_final` от параметра `b`

```
p_b_saturation = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_saturation,
            sub.b,
            sub.saturation_final,
            seriestype = :scatter,
            label = "$model_type_string: saturation_final от b"
        )
    end
end

plot!(
    p_b_saturation,
    xlabel = "b",
    ylabel = "n_final / N",
    title = "Зависимость финального насыщения от параметра b",
    legend = :bottomright,
```

```

        grid = true
    )

    savefig(p_b_saturation, plotsdir(script_name, "saturation_final_vs_b.png"))

```

5.27 Бенчмаркинг

```

println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        N = params[:N]
        n0 = params[:n0]

        u0 = [n0]
        p = (a = params[:a], b = params[:b], N = N)

        model! = choose_model(params[:model_type])

        prob = ODEProblem(model!, u0, params[:tspan], p)

        return solve(
            prob,

```

```

        params[:solver];
        saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
    )
end

println(
    "\nБенчмарк для model_type=$(params[:model_type]), ",
    "a=$(params[:a]), b=$(params[:b]):"
)

bmark = @benchmark $benchmark_run() samples = 80 evals = 1
tsec = median(bmark).time / 1e9

println(" Медианное время: ", round(tsec; digits = 6), " сек")

push!(
    benchmark_results,
    (
        model_type = string(params[:model_type]),
        a = params[:a],
        b = params[:b],
        time = tsec
    )
)
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)

```

5.28 График времени вычисления от параметра a

```
p_time_a = plot(size = (900, 520), dpi = 150)

for model_type_string in unique(bench_df.model_type)
    sub = bench_df[bench_df.model_type == model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_time_a,
            sub.a,
            sub.time,
            seriestype = :scatter,
            label = "$model_type_string: время от a"
        )
    end
end

plot!(
    p_time_a,
    xlabel = "a",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметра a",
    legend = :topright,
    grid = true
)

savefig(p_time_a, plotsdir(script_name, "computation_time_vs_a.png"))
```

5.29 График времени вычисления от параметра b

```
p_time_b = plot(size = (900, 520), dpi = 150)

for model_type_string in unique(bench_df.model_type)
    sub = bench_df[bench_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_time_b,
            sub.b,
            sub.time,
            seriestype = :scatter,
            label = "$model_type_string: время от b"
        )
    end
end

plot!(
    p_time_b,
    xlabel = "b",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметра b",
    legend = :topright,
    grid = true
)

savefig(p_time_b, plotsdir(script_name, "computation_time_vs_b.png"))
```

5.30 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_experiments base_results param_gr

@save datadir(script_name, "all_plots.jld2") p_a_final p_b_final p_a_max p_b_max p_a_

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

println("\nРезультаты сохранены в:")
println(" • data/${script_name}/single/ - базовые эксперименты")
println(" • data/${script_name}/parametric_scan/ - параметрическое сканирование")
println(" • data/${script_name}/table_single_case1.csv - таблица для первой модели")
println(" • data/${script_name}/table_single_case2.csv - таблица для второй модели")
println(" • data/${script_name}/table_single_case3.csv - таблица для третьей модели")
println(" • data/${script_name}/results_summary.csv - сводная таблица")
println(" • data/${script_name}/timeseries_full.csv - полные временные ряды")
println(" • data/${script_name}/benchmark_results.csv - результаты бенчмаркинга")
println(" • data/${script_name}/all_results.jld2 - сводные данные")
println(" • data/${script_name}/all_plots.jld2 - объекты графиков")
println(" • plots/${script_name}/ - все графики")
```

5.31 Базовые эксперименты

5.31.1 Первая модель (model_type = case1)

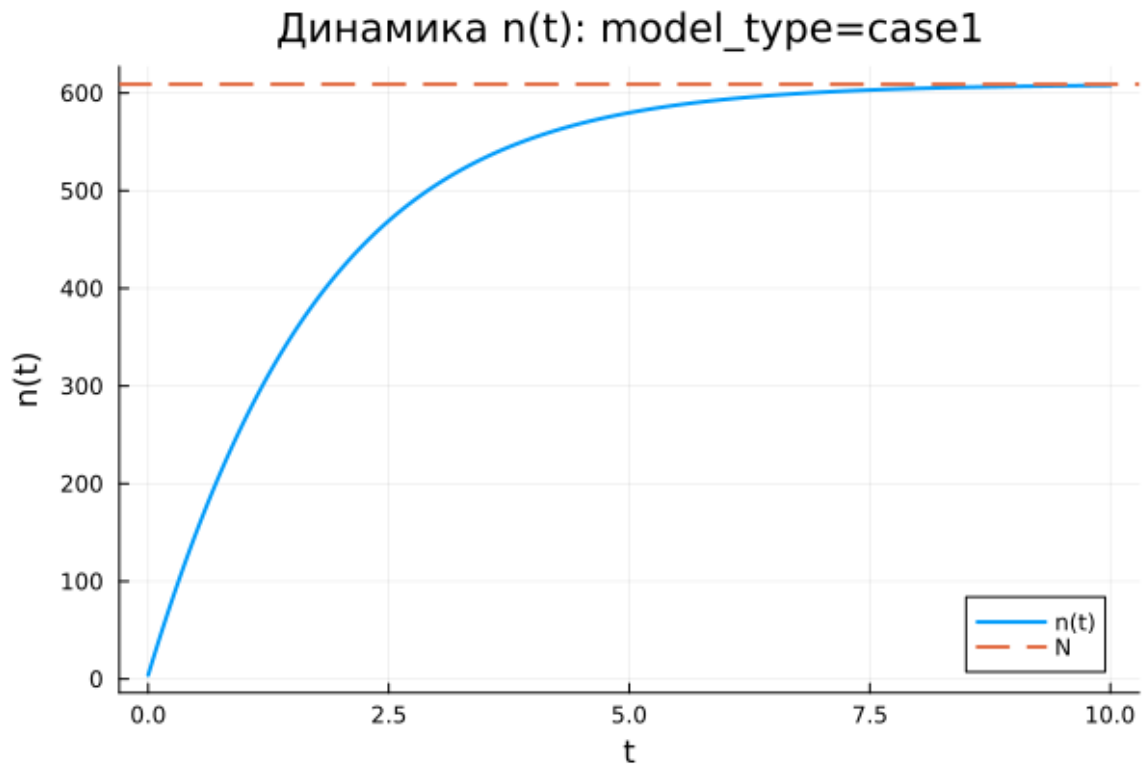


Рисунок 5.1: Первая модель: зависимость $n(t)$

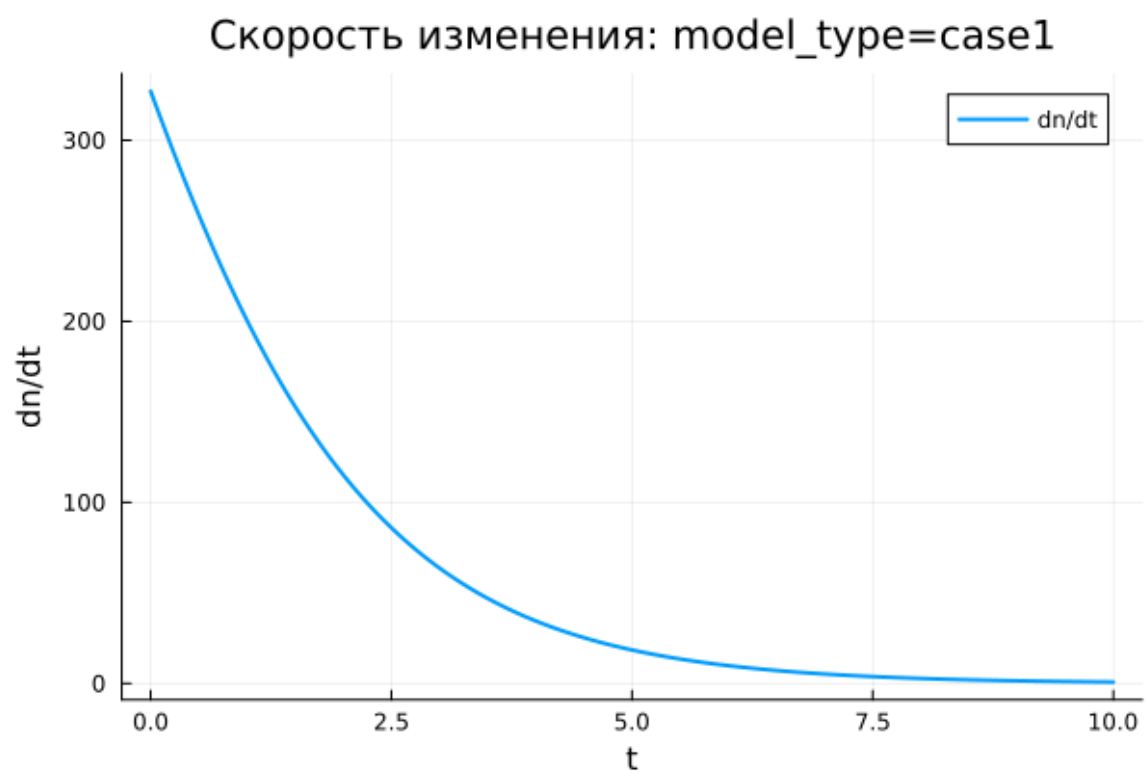


Рисунок 5.2: Первая модель: скорость изменения

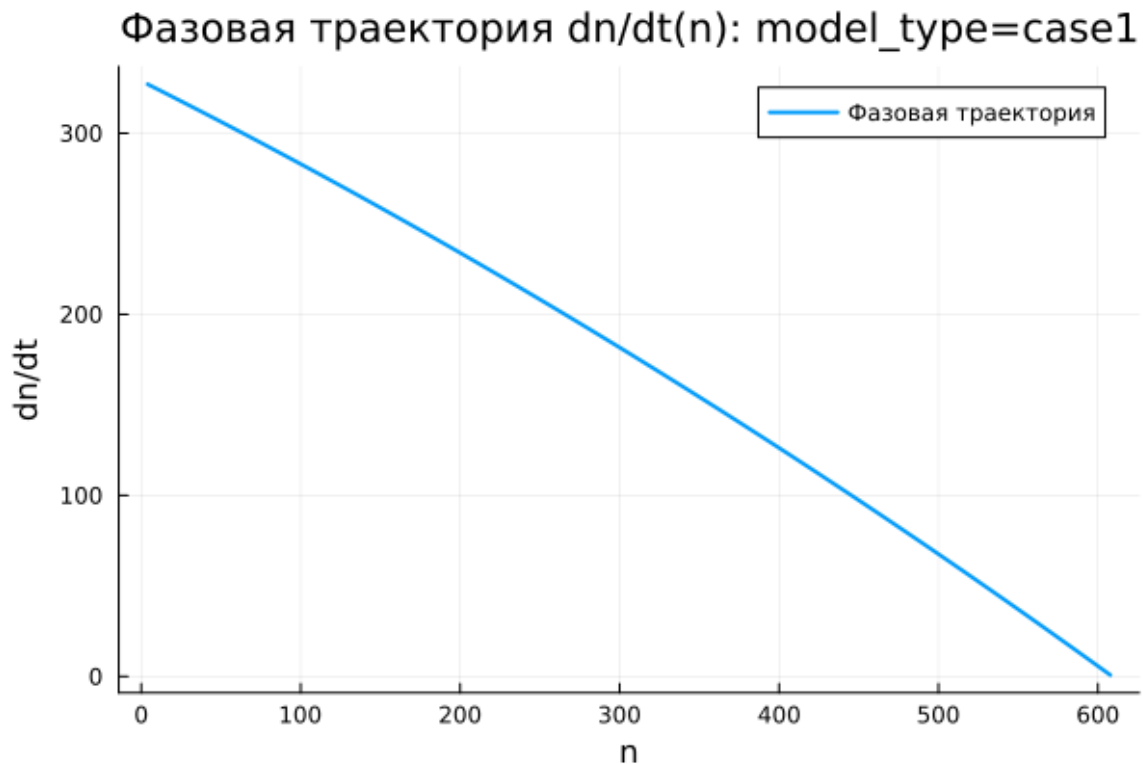


Рисунок 5.3: Первая модель: фазовая траектория

В первой модели функция $n(t)$ возрастает монотонно. В начале процесса число информированных покупателей мало по сравнению с предельным уровнем $N = 609$, поэтому реклама быстро увеличивает охват аудитории. Далее, по мере приближения $n(t)$ к N , рост становится менее интенсивным.

На графике динамики видно, что к концу интервала $t \in [0; 10]$ решение почти выходит на уровень насыщения. Горизонтальная пунктирная линия показывает значение N , к которому стремится численная траектория. Функция $n(t)$ не превышает этот уровень, а приближается к нему снизу.

График производной показывает, что наибольшая скорость роста достигается в начальный момент времени. Затем величина $\frac{dn}{dt}$ убывает и стремится к нулю. Следовательно, основной прирост числа информированных покупателей происходит в начале кампании, а затем система постепенно переходит к насыщенному состоянию.

Фазовая траектория $\frac{dn}{dt}(n)$ подтверждает этот результат. При малых значениях n скорость изменения максимальна, а при приближении к N она уменьшается почти до нуля. Траектория имеет убывающий характер, поэтому первая модель описывает быстрый начальный рост с последующим замедлением.

Значение $n = N$ является равновесным состоянием модели. При достижении насыщения множитель $(N - n)$ обращается в ноль, поэтому дальнейший рост прекращается.

5.31.2 Вторая модель (model_type = case2)

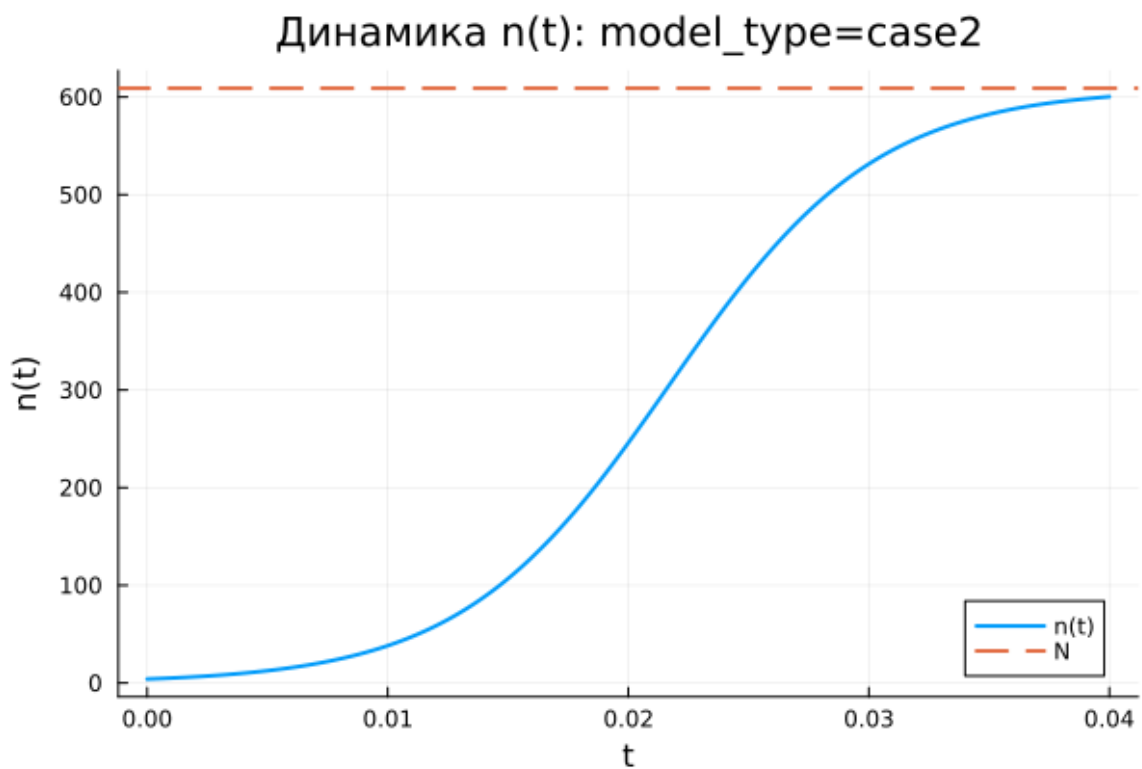


Рисунок 5.4: Вторая модель: зависимость $n(t)$

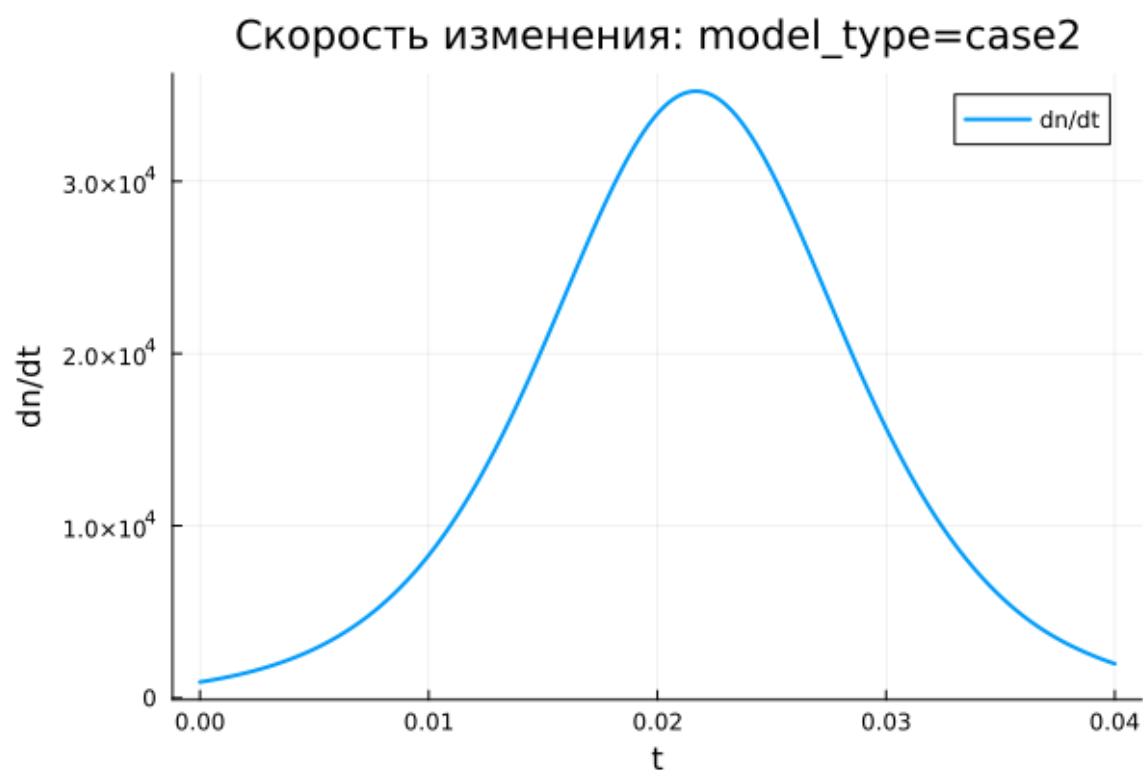


Рисунок 5.5: Вторая модель: скорость изменения

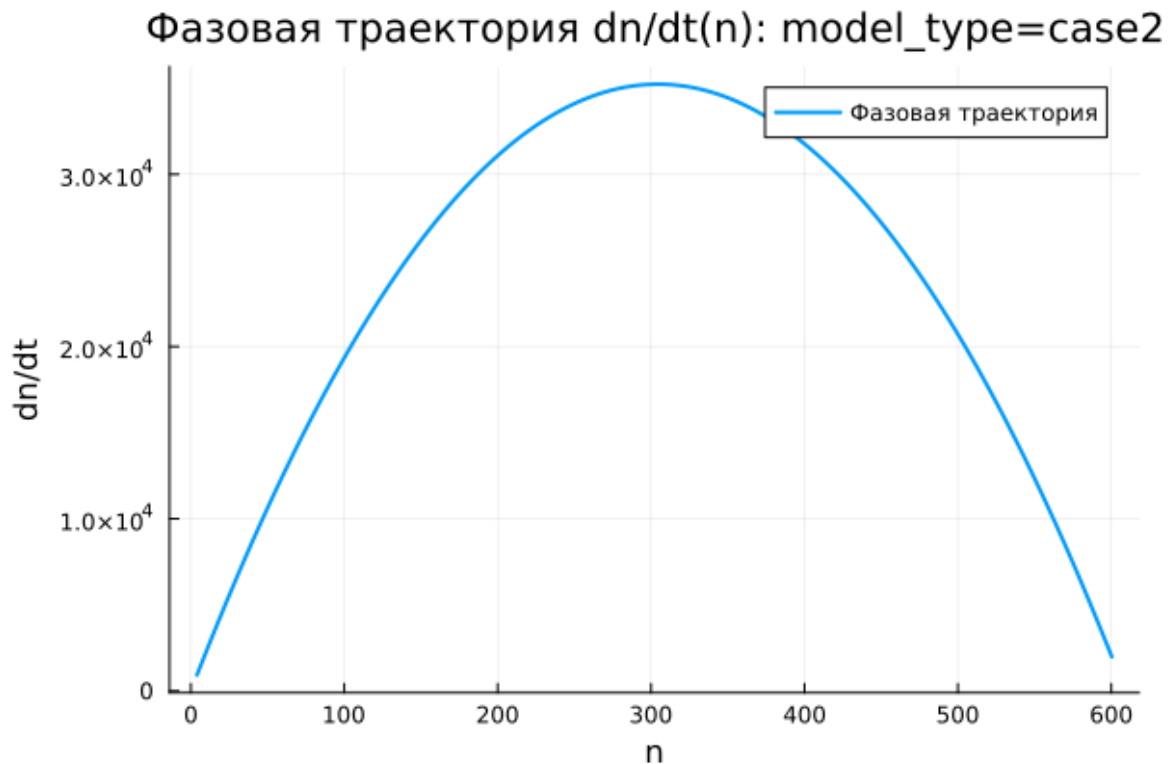


Рисунок 5.6: Вторая модель: фазовая траектория

Во второй модели число информированных покупателей растёт заметно быстрее, чем в первой. Расчёт проводится на коротком интервале $t \in [0; 0,04]$, однако за это время решение почти достигает уровня $N = 609$. Причина состоит в большом значении коэффициента b , который усиливает нелинейное слагаемое $bn(t)$.

График $n(t)$ имеет выраженную S-образную форму. В начале рост умеренный, поскольку значение n ещё мало. Затем скорость резко увеличивается. После этого при приближении к уровню насыщения рост снова замедляется.

График $\frac{dn}{dt}$ содержит один отчётливый максимум. Сначала скорость распространения рекламы возрастает, затем достигает пикового значения, после чего быстро уменьшается почти до нуля. Это означает, что модель описывает режим самоускоряющегося распространения информации, ограниченного конечным размером аудитории.

Фазовая траектория $\frac{dn}{dt}(n)$ имеет вид, близкий к параболе. При небольшом числе информированных покупателей скорость невелика. Затем она возрастает и достигает максимума примерно при $n \approx 304,5$, после чего снижается при движении к N . Здесь одновременно действуют два фактора: усиление распространения через множитель bn и торможение через множитель $(N - n)$.

Для второго случая максимум скорости можно найти аналитически. Правая часть уравнения имеет вид

$$f(n) = (a + bn)(N - n),$$

где

$$a = 0.000021, \quad b = 0.38, \quad N = 609.$$

Максимум скорости достигается в точке, где производная $f'(n)$ равна нулю:

$$f'(n) = b(N - n) - (a + bn) = bN - a - 2bn.$$

Отсюда

$$n_* = \frac{bN - a}{2b}.$$

После подстановки параметров получаем

$$n_* = \frac{0.38 \cdot 609 - 0.000021}{2 \cdot 0.38} \approx 304.5.$$

Для нахождения соответствующего момента времени используем решение уравнения

$$\frac{dn}{dt} = (a + bn)(N - n).$$

После разделения переменных получаем

$$\frac{a + bn}{N - n} = \frac{a + bn_0}{N - n_0} e^{(a+bN)t},$$

где $n_0 = 4$. Тогда момент достижения максимальной скорости равен

$$t_* = \frac{1}{a + bN} \ln \left(\frac{a + bn_*}{N - n_*} \cdot \frac{N - n_0}{a + bn_0} \right).$$

Подстановка численных значений даёт

$$t_* \approx 0.02169.$$

Следовательно, во второй модели максимальная скорость распространения рекламы достигается примерно в момент времени

$$t \approx 0.0217.$$

Вторая модель описывает наиболее резкий переход к насыщению. В отличие от первой модели, максимум скорости возникает не в начале процесса, а внутри траектории, когда аудитория уже частично информирована, но до предельного уровня ещё остаётся значительный запас.

5.31.3 Третья модель (model_type = case3)

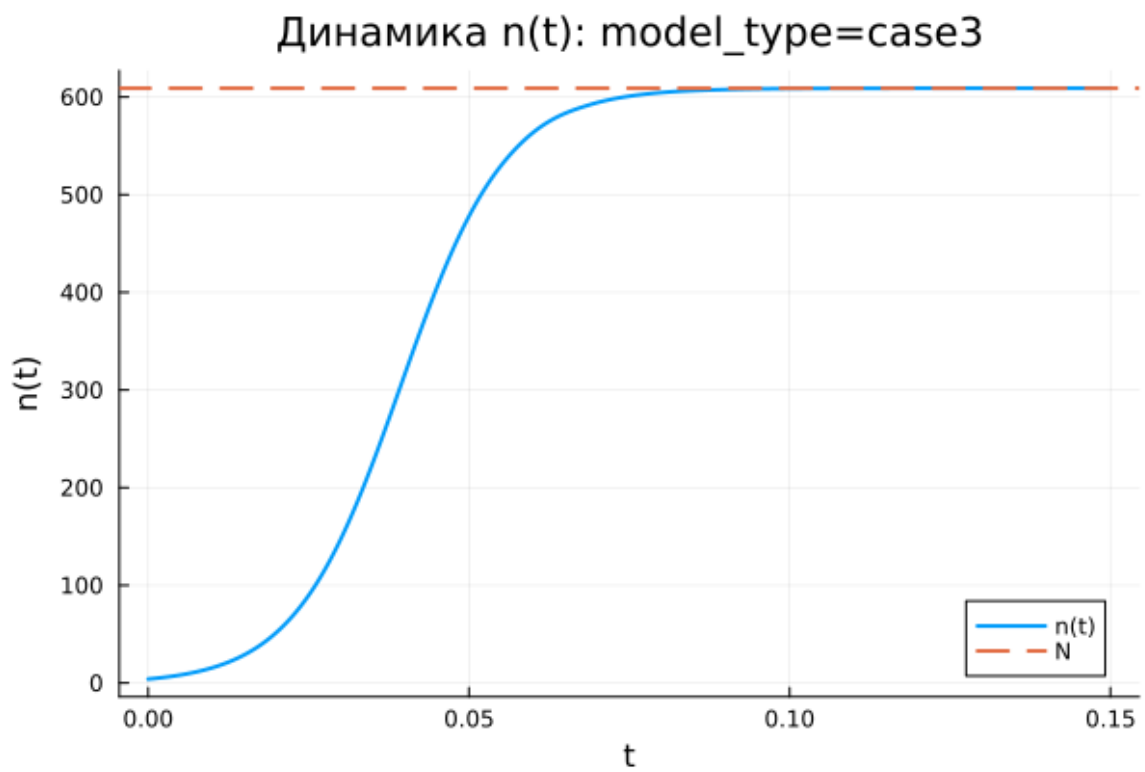


Рисунок 5.7: Третья модель: зависимость $n(t)$

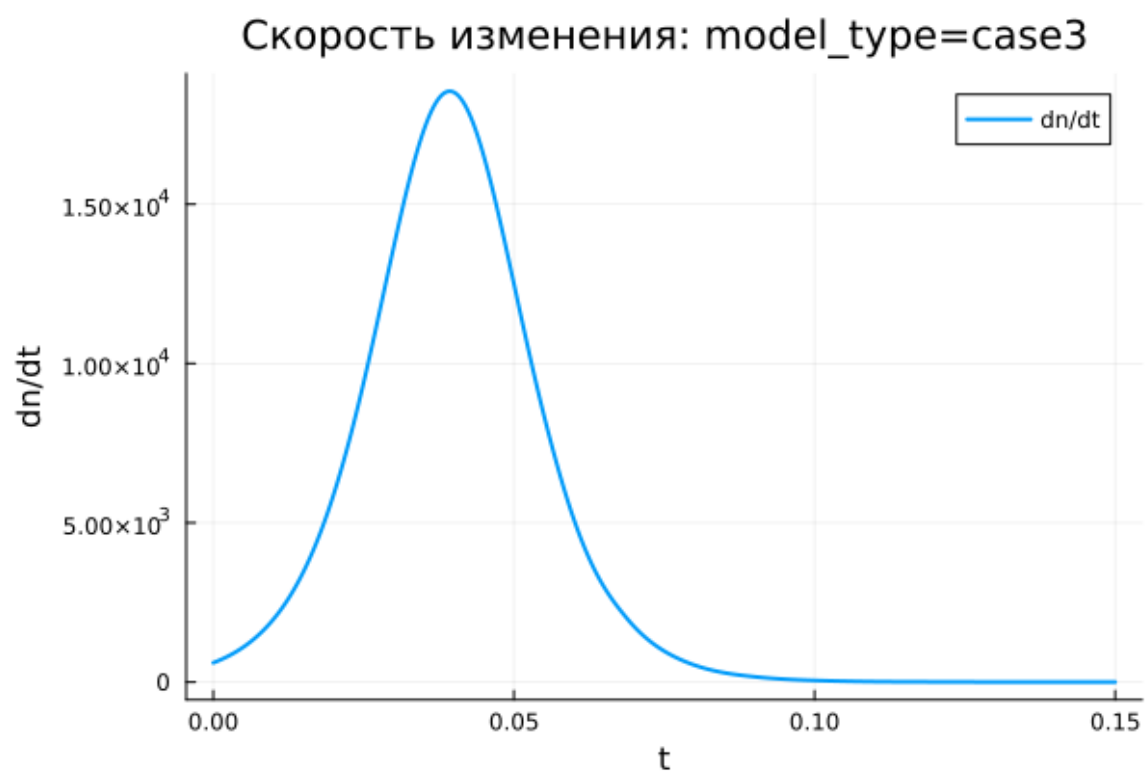


Рисунок 5.8: Третья модель: скорость изменения

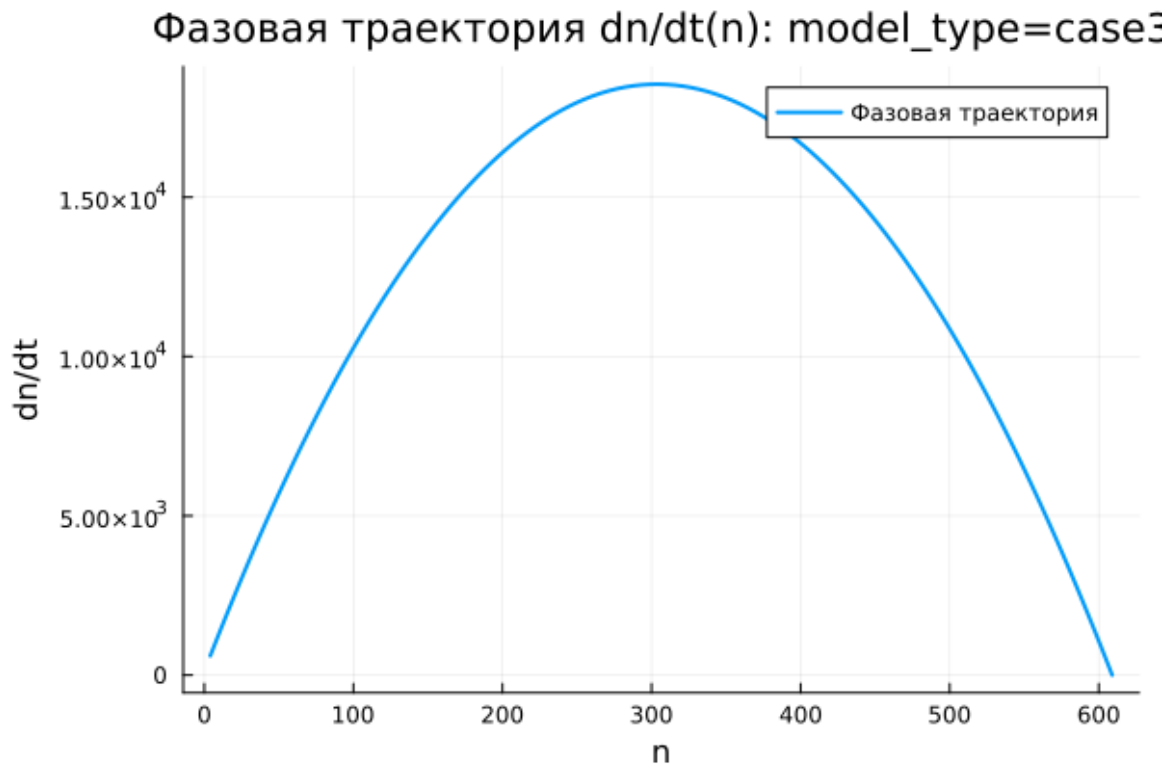


Рисунок 5.9: Третья модель: фазовая траектория

В третьей модели функция $n(t)$ также возрастает и быстро приближается к уровню $N = 609$. На графике видно, что основной прирост происходит в начале интервала $t \in [0; 0,15]$. После этого кривая почти совпадает с горизонтальным уровнем насыщения.

Отличие третьей модели состоит в наличии периодических коэффициентов $\cos t$ и $\cos(2t)$. На выбранном коротком интервале времени эти функции остаются положительными и близкими к единице, поэтому они не вызывают колебательного режима, а лишь меняют интенсивность роста.

График скорости $\frac{dn}{dt}$ содержит один выраженный пик. Сначала скорость увеличивается, затем достигает максимума, после чего быстро убывает к нулю. После выхода $n(t)$ на уровень насыщения дальнейшее изменение практически прекращается.

Фазовая траектория $\frac{dn}{dt}(n)$ по форме близка к параболической. При малом

числе информированных покупателей скорость невелика, затем она возрастает и достигает максимума в средней части диапазона. При дальнейшем приближении к N множитель $(N - n)$ уменьшает скорость почти до нуля.

Третья модель описывает насыщаемый рост с коэффициентами, зависящими от времени. На рассматриваемом интервале периодические множители не меняют общий характер динамики: решение остаётся монотонным, достигает предельного уровня и переходит в равновесное состояние.

5.32 Сравнение базовых экспериментов

Во всех трёх моделях функция $n(t)$ стремится к одному предельному уровню $N = 609$. Это связано с множителем $(N - n)$, который уменьшает скорость роста при приближении к насыщению и полностью останавливает изменение при $n = N$.

Первая модель характеризуется быстрым началом и дальнейшим плавным замедлением. Максимум скорости наблюдается сразу после старта, а фазовая траектория имеет убывающий вид.

Вторая модель демонстрирует наиболее резкий S-образный рост. Скорость сначала увеличивается, затем достигает максимума и после этого уменьшается. Фазовая траектория имеет выраженную параболическую форму, что указывает на наличие внутренней точки максимального роста.

Третья модель по характеру близка ко второй, но включает временную зависимость коэффициентов. На выбранном интервале эта зависимость не приводит к колебаниям, поскольку $\cos t$ и $\cos(2t)$ сохраняют положительные значения.

Все три модели приводят систему к насыщению, однако различаются скоростью перехода, формой графика $\frac{dn}{dt}$ и положением максимума скорости.

5.33 Параметрическое сканирование

5.33.1 Зависимость итогового значения n от параметра a

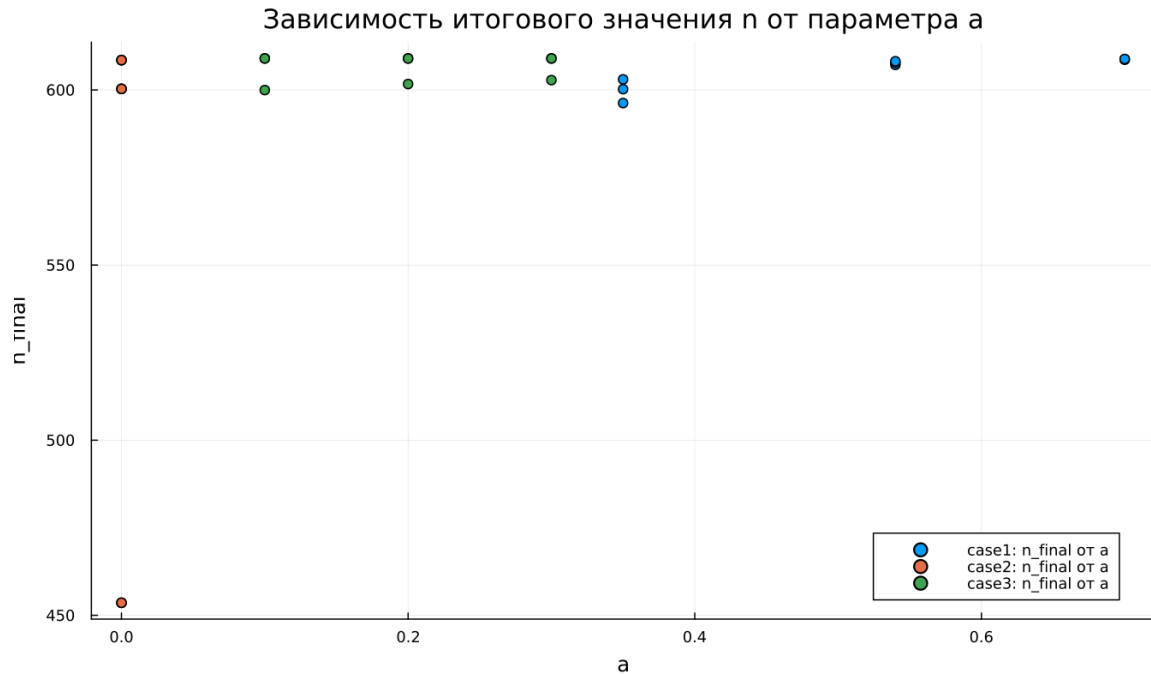


Рисунок 5.10: Зависимость итогового значения n от параметра a

На графике показана зависимость итогового значения n_{final} от параметра a для трёх моделей. Большинство точек находится около уровня $N = 609$, что говорит о почти полном насыщении к концу расчётного интервала.

Для первой модели значения n_{final} близки к N при всех рассмотренных значениях a . Следовательно, изменение параметра a в выбранном диапазоне не нарушает общий сценарий: решение продолжает стремиться к предельному уровню.

Во второй модели присутствует заметное отклонение при малом значении a . В этом эксперименте итоговое значение n существенно ниже N и составляет примерно $n_{final} \approx 455$. Это означает, что при выбранной комбинации параметров процесс не успевает завершить переход к насыщению за короткий

расчётный интервал.

Для третьей модели значения n_{final} также находятся вблизи N . Периодические коэффициенты не приводят к заметному уменьшению итогового уровня при выбранных параметрах.

В целом параметр a влияет не только на конечное значение, но и на скорость приближения к насыщению. При малой интенсивности роста система может не успеть достичь уровня N за заданное время моделирования.

5.33.2 Зависимость итогового значения n от параметра b

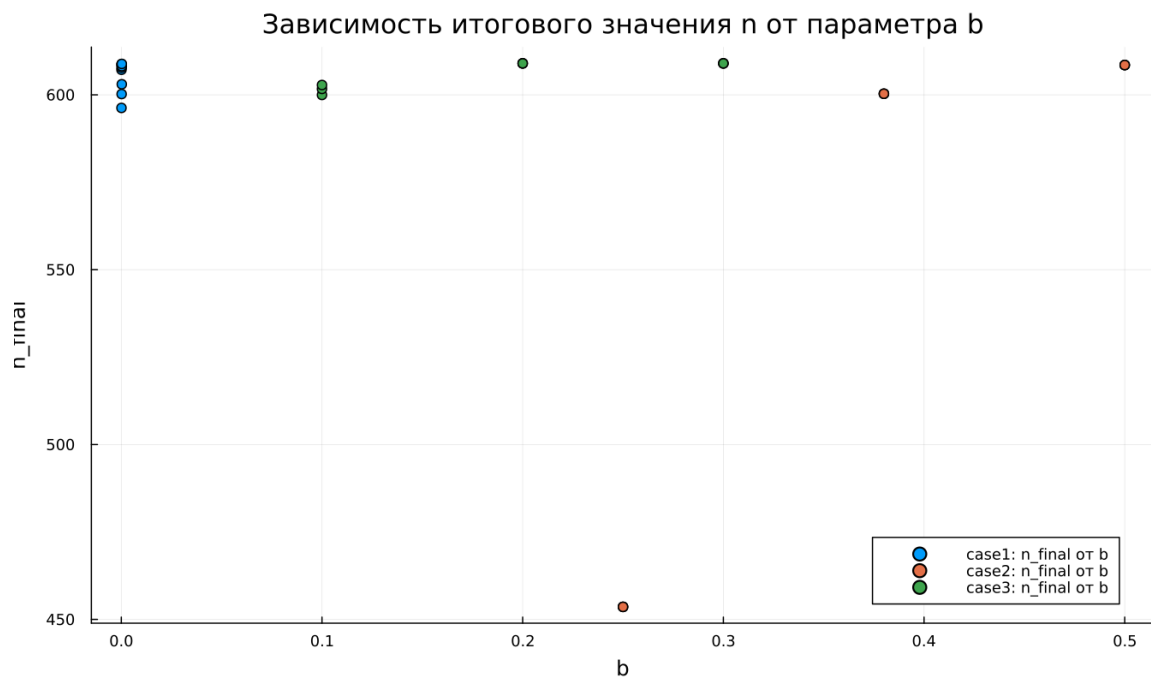


Рисунок 5.11: Зависимость итогового значения n от параметра b

График зависимости n_{final} от параметра b показывает, что в большинстве экспериментов итоговое значение также расположено около $N = 609$. Это подтверждает устойчивое стремление решений к насыщению.

Для первой модели точки сгруппированы около малых значений b , поскольку при сканировании использовался небольшой диапазон этого коэффициен-

та. Несмотря на это, итоговые значения n остаются близкими к предельному уровню.

Во второй модели влияние параметра b выражено сильнее. При одном из значений коэффициента итоговое значение n заметно ниже уровня насыщения. Это показывает, что для соответствующей комбинации параметров рост оказывается недостаточно быстрым, поэтому система не успевает приблизиться к N за время расчёта.

Для третьей модели точки расположены около N , что указывает на сохранение режима насыщаемого роста. Даже при наличии множителей $\cos t$ и $\cos(2t)$ итоговое значение остаётся близким к верхнему пределу.

График показывает, что параметр b особенно важен для моделей с нелинейным усилением роста через слагаемое bn . При недостаточном значении коэффициента выход к насыщению замедляется.

5.33.3 Зависимость максимального значения n от параметра a

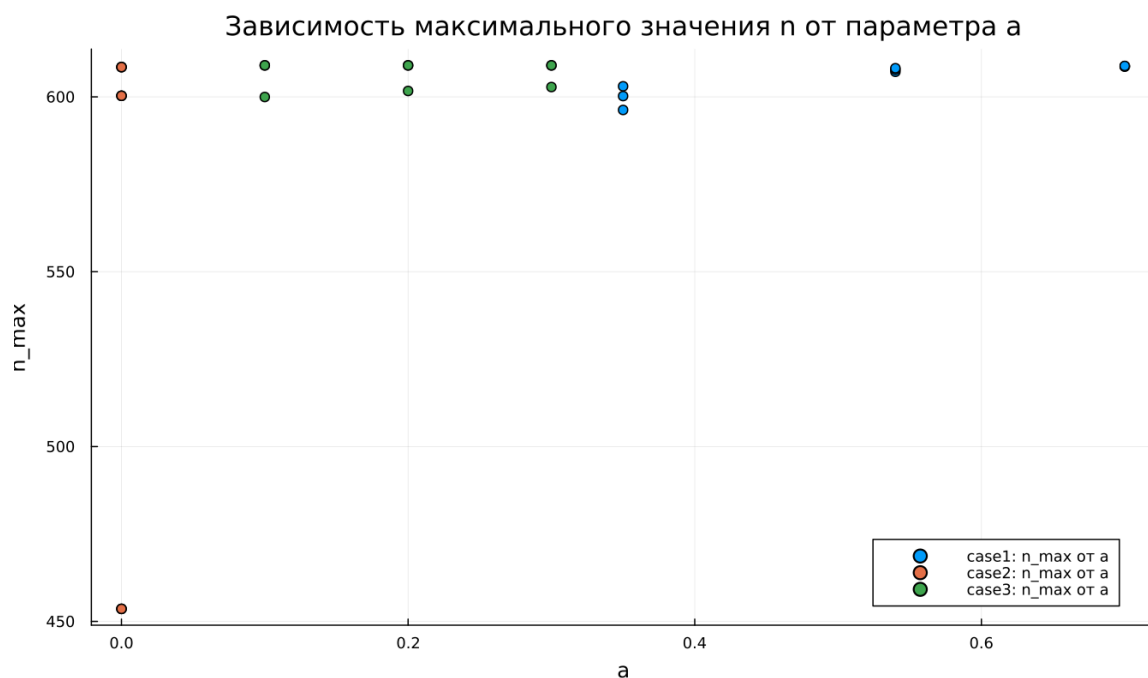


Рисунок 5.12: Зависимость максимального значения n от параметра a

На графике представлена зависимость максимального значения n_{max} от параметра a . Поскольку в базовых экспериментах функция $n(t)$ возрастает монотонно, величина n_{max} почти совпадает с итоговым значением n_{final} .

Для первой модели максимальные значения находятся около уровня $N = 609$. Это показывает, что решение достигает верхней границы или подходит к ней очень близко.

Во второй модели снова выделяется точка с меньшим значением n_{max} . На всём временном интервале решение не успевает приблизиться к насыщению. Причина связана не с изменением направления движения, а с недостаточной скоростью роста на выбранном интервале.

Для третьей модели максимальные значения сохраняются около N . Следовательно, коэффициенты, зависящие от времени, не препятствуют достижению высокого уровня n при выбранном диапазоне параметров.

Поскольку n_{max} отражает наибольшее значение решения за время расчёта, этот график подтверждает выводы, полученные при анализе n_{final} .

5.33.4 Зависимость максимального значения n от параметра b

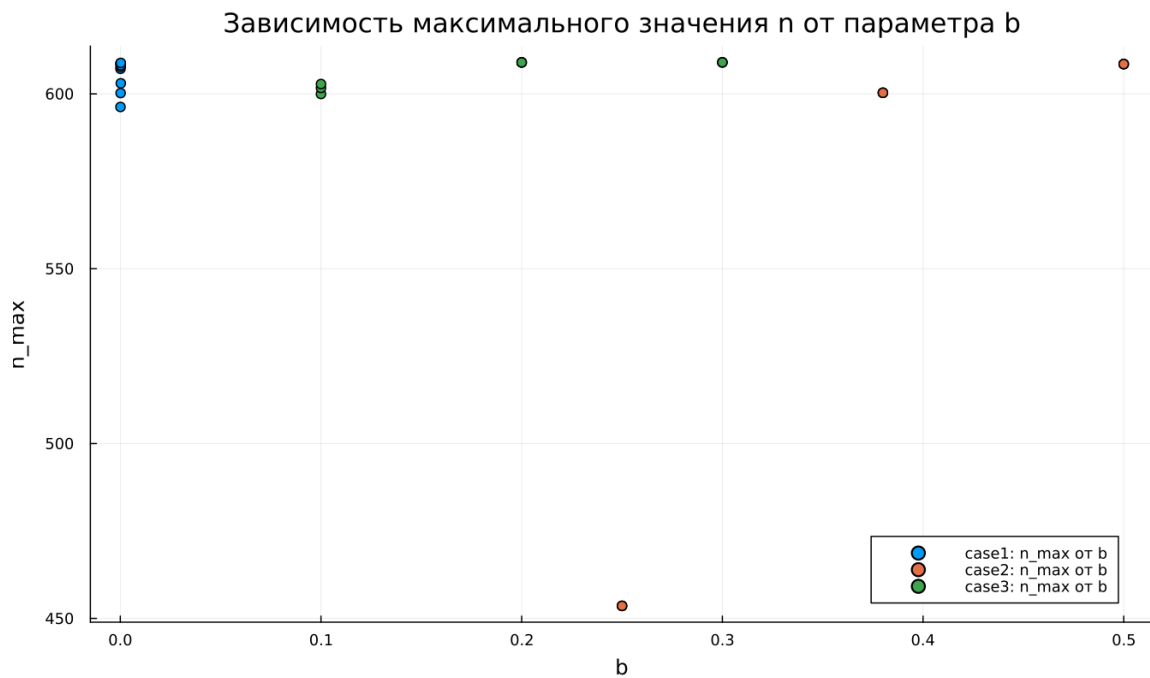


Рисунок 5.13: Зависимость максимального значения n от параметра b

График зависимости n_{max} от параметра b имеет структуру, близкую к графику $n_{final}(b)$. Это связано с монотонным ростом решений: максимум достигается в конце расчётного интервала или рядом с ним.

Для первой модели значения n_{max} сосредоточены около N . Изменение параметра b в малом диапазоне не приводит к заметному снижению максимального уровня.

Во второй модели одна точка расположена значительно ниже остальных. Это показывает, что при соответствующей комбинации параметров модель не выходит на насыщение. Остальные эксперименты для второй модели дают значения около верхнего предела.

Для третьей модели все точки остаются вблизи N , что говорит о стабильном достижении насыщения. Параметр b влияет на скорость роста, но в проведённых экспериментах не меняет конечный характер динамики.

График подтверждает, что максимальный уровень n ограничивается величиной N , а параметры a и b определяют скорость приближения к этому пределу.

5.33.5 Зависимость финального насыщения от параметра a

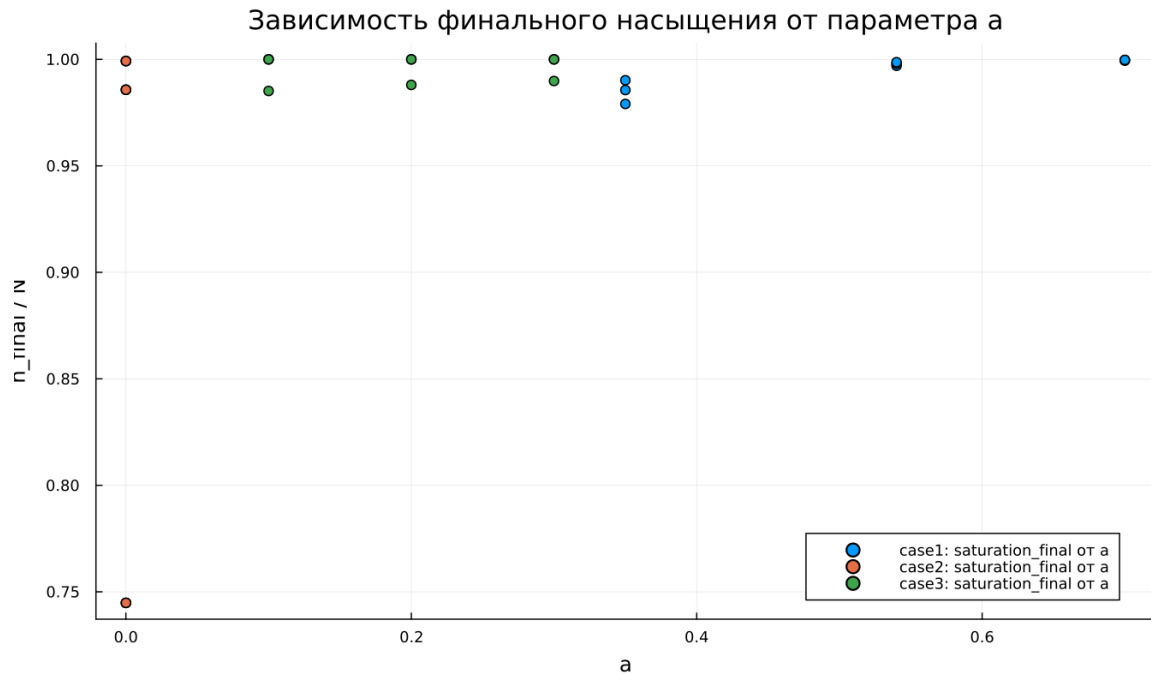


Рисунок 5.14: Зависимость финального насыщения от параметра a

На графике показана зависимость относительного итогового насыщения $\frac{n_{final}}{N}$ от параметра a . Значение, близкое к 1, означает почти полное достижение предельного уровня N .

Для первой модели насыщение находится около 1 при всех рассмотренных значениях a . Это показывает, что модель устойчиво достигает предельного состояния.

Во второй модели наблюдается одна точка с насыщением около 0,75. В этом эксперименте итоговое значение составляет примерно три четверти от возможного максимума. Остальные точки расположены значительно выше и соответствуют почти полному насыщению.

Для третьей модели значения $\frac{n_{final}}{N}$ находятся вблизи 1. Это указывает на почти полный выход системы к предельному уровню даже при временной зависимости коэффициентов.

График удобен для сопоставления моделей, поскольку нормировка на N позволяет оценивать не абсолютное значение n , а степень приближения к верхней границе.

5.33.6 Зависимость финального насыщения от параметра b

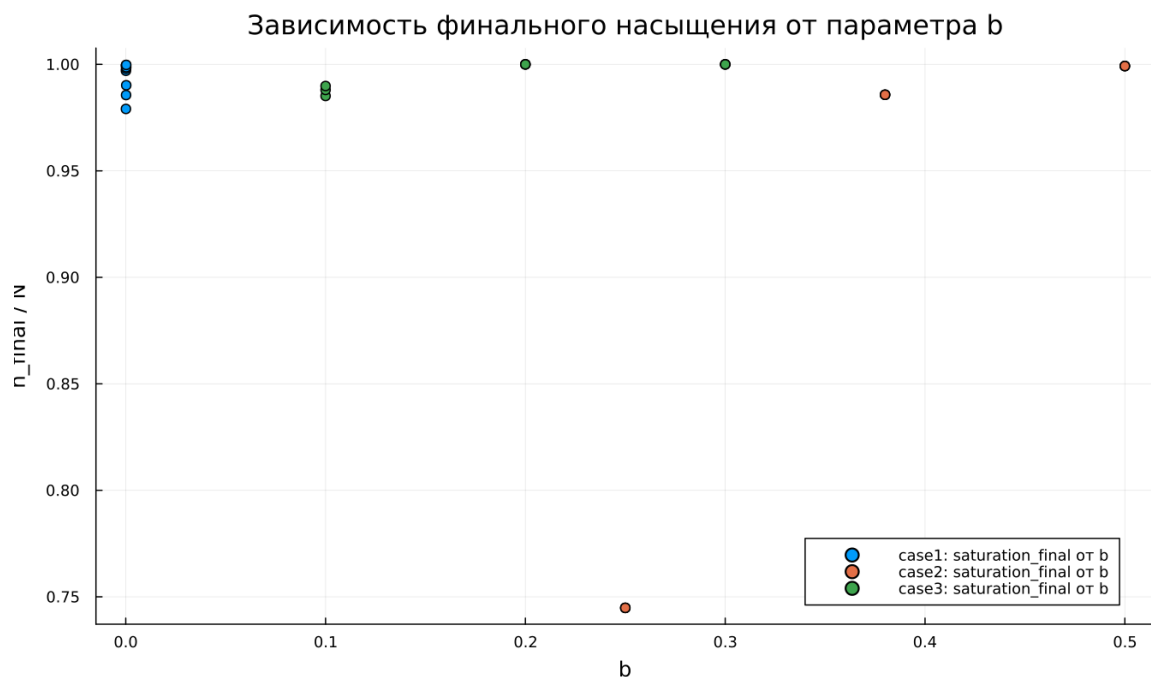


Рисунок 5.15: Зависимость финального насыщения от параметра b

График зависимости $\frac{n_{final}}{N}$ от параметра b показывает, что большинство экспериментов завершается почти полным насыщением. Значения расположены вблизи 1, что соответствует достижению уровня N .

Для первой модели насыщение остаётся высоким при всех рассмотренных значениях b . Даже малые значения параметра не мешают решению приблизиться к предельному уровню за выбранное время.

Во второй модели снова видно одно пониженное значение. В этом эксперименте система достигает только около 75% от уровня N . Этот результат отражает чувствительность второй модели к сочетанию параметров и длине расчётного интервала.

Для третьей модели точки расположены около полного насыщения. Временные множители не создают заметного уменьшения итогового уровня.

В целом график подтверждает стремление системы к насыщению. Однако при отдельных наборах параметров времени моделирования оказывается недостаточно для полного выхода на уровень N .

5.34 Бенчмаркинг

5.34.1 Зависимость времени решения ODE от параметра a

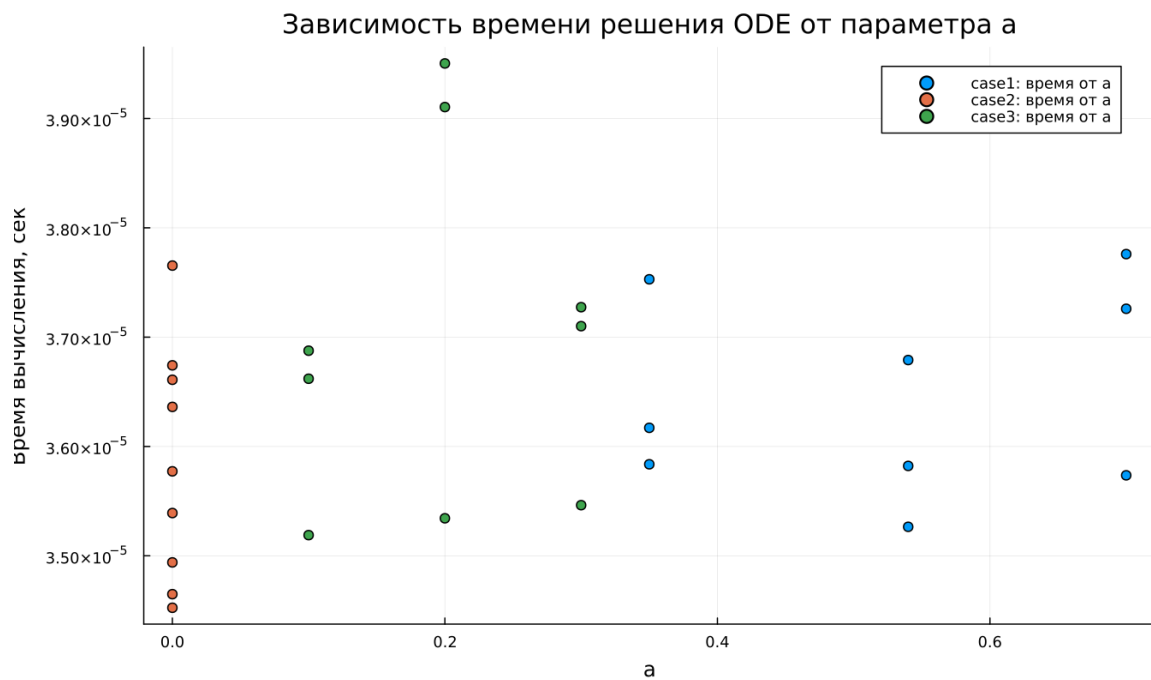


Рисунок 5.16: Зависимость времени решения ODE от параметра a

На графике показано медианное время численного решения задачи ОДУ при различных значениях параметра a . Время вычислений находится примерно в диапазоне от $3,45 \cdot 10^{-5}$ до $3,95 \cdot 10^{-5}$ секунды.

Для всех трёх моделей значения времени близки друг к другу. Это означает, что рассматриваемые задачи имеют малую вычислительную сложность, а изменение параметров не приводит к резкому увеличению затрат на интегрирование.

Явной монотонной зависимости времени решения от параметра a не наблюдается. Точки имеют небольшой разброс, который может быть связан с особенностями работы численного метода, кэшированием, системной нагрузкой и погрешностью измерения при очень малых временах выполнения.

Третья модель в отдельных точках требует немного больше времени. Вероятная причина состоит в более сложной правой части уравнения, где дополнительно вычисляются функции $\cos t$ и $\cos(2t)$.

В целом бенчмаркинг показывает, что все три модели решаются быстро, а различия между ними по времени вычислений остаются незначительными.

5.34.2 Зависимость времени решения ODE от параметра b

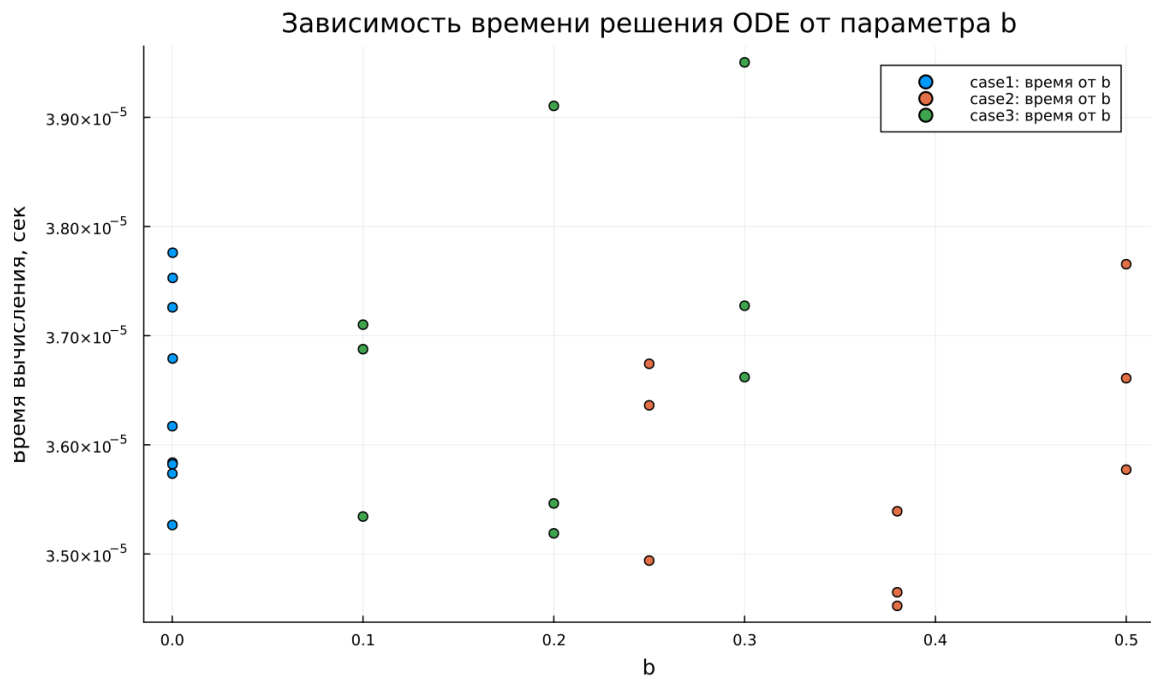


Рисунок 5.17: Зависимость времени решения ODE от параметра b

На графике представлена зависимость времени решения ОДУ от параметра b . Как и при изменении параметра a , значения времени находятся в узком диапазоне порядка 10^{-5} секунды.

Для первой модели точки сосредоточены около малых значений b , что соответствует выбранной сетке параметров. Время решения немного колеблется, но не демонстрирует устойчивого роста или убывания.

Для второй модели значения времени также изменяются с небольшим разбросом. Даже при увеличении b вычислительные затраты не возрастают существенно. Это говорит о том, что выбранный численный метод справляется с задачей без заметного усложнения интегрирования.

Для третьей модели часть точек расположена выше, чем у первых двух моделей. Вероятная причина заключается в более сложной формуле правой части, содержащей тригонометрические множители. При этом различие остаётся

небольшим и не меняет общий вывод о высокой скорости расчёта.

График показывает, что параметр b сильнее влияет на динамику решения, чем на время вычислений. В рамках проведённого сканирования вычислительная стоимость остаётся стабильной для всех трёх моделей.

5.35 Выводы

1. Первая модель (case1) описывает насыщаемый рост функции $n(t)$. Решение монотонно увеличивается и постепенно приближается к предельному уровню $N = 609$, не превышая его.
2. Во второй модели (case2) наблюдается наиболее быстрый рост. Функция $n(t)$ имеет выраженную S-образную форму: сначала рост идёт медленно, затем ускоряется, а после этого замедляется при приближении к насыщению.
3. Для второй модели максимум скорости распространения рекламы достигается при $n_* \approx 304,5$. Соответствующий момент времени равен $t_* \approx 0,0217$.
4. Третья модель (case3) также приводит систему к насыщению, но отличается коэффициентами, зависящими от времени. На выбранном интервале множители $\cos t$ и $\cos(2t)$ не создают колебаний, поэтому решение остаётся монотонным.
5. Во всех трёх моделях предельное значение N играет роль уровня насыщения. При приближении $n(t)$ к N множитель $(N - n)$ уменьшает скорость роста, а при $n = N$ правая часть уравнения становится равной нулю.
6. Графики скорости $\frac{dn}{dt}$ показывают различия между моделями. В первой модели скорость максимальна в начале процесса и затем убывает. Во

второй и третьей моделях скорость сначала возрастает, достигает максимума и после этого снижается почти до нуля.

7. Фазовые траектории $\frac{dn}{dt}(n)$ подтверждают характер динамики. Для первой модели траектория убывает, а для второй и третьей моделей имеет форму, близкую к параболической, с внутренней точкой максимальной скорости роста.
8. Параметры a и b в первую очередь влияют на скорость выхода к насыщению. При достаточно больших значениях параметров система быстро достигает уровня N , а при отдельных комбинациях, особенно во второй модели, переход к насыщению не завершается за заданное время.
9. Метрики n_{final} и n_{max} в большинстве экспериментов близки к $N = 609$, что указывает на достижение предельного состояния. Заметные отклонения связаны с недостаточной длительностью расчётного интервала для некоторых наборов параметров.
10. Метрика $\text{saturation_final} = \frac{n_{final}}{N}$ позволяет оценить степень достижения насыщения. Значения, близкие к единице, соответствуют почти полному выходу системы на предельный уровень, а пониженные значения показывают незавершённый переходный процесс.
11. Бенчмаркинг показал, что численное решение всех трёх моделей выполняется быстро. Время вычислений имеет порядок 10^{-5} секунды, а изменение параметров a и b почти не влияет на вычислительные затраты.
12. Третья модель немного сложнее с вычислительной точки зрения, поскольку правая часть содержит тригонометрические функции. Однако разница во времени решения остаётся малой и не оказывает существенного влияния на общую эффективность расчётов.

13. Все рассмотренные модели описывают процесс насыщаемого роста, но отличаются скоростью перехода к предельному состоянию и формой зависимости $\frac{dn}{dt}$ от времени и от значения n .

Список литературы

1. Модель Мальтуса
2. Логистическая модель роста